

CI/CD Deployment Guide: AWS EC2, Jenkins, SonarQube, Docker, and Tomcat

Once you have perfected the Bus Tracker project locally, follow this comprehensive guide to set up a robust, automated deployment pipeline. This guide assumes you want to package your Spring Boot backend as a traditional `.war` file and deploy it to a dedicated Tomcat Docker container.

1. Project Code Preparation

By default, Spring Boot creates an executable `.jar` with an embedded Tomcat. Since you want to use a standalone Docker Tomcat container, you need to prepare the backend project to build a `.war` file.

Update pom.xml

Add the packaging type and mark the embedded Tomcat dependency as provided:

```
<packaging>war</packaging>

<dependencies>
  <!-- Add this to prevent embedded Tomcat from conflicting with the standalone Tomcat -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Update the Main Application Class

Ensure your `BusTrackerApplication.java` extends `SpringBootServletInitializer`:

```
package com.college.bus;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

@SpringBootApplication
public class BusTrackerApplication extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        return application.sources(BusTrackerApplication.class);
    }

    public static void main(String[] args) {
        SpringApplication.run(BusTrackerApplication.class, args);
    }
}
```

2. Infrastructure Setup (AWS EC2)

1. Launch an EC2 Instance:

- Use an **Ubuntu 22.04 LTS** (or Amazon Linux 2023) instance.
- Recommended Size: **t3.medium** or larger (Jenkins + SonarQube + Build processes consume significant memory).

2. Configure Security Group:

- **SSH (22)**: For your access.
- **HTTP (80) / HTTPS (443)**: For the frontend application.
- **Jenkins (8080)**: For the Jenkins UI.
- **SonarQube (9000)**: For the SonarQube UI.

- **Tomcat Application (8081):** For the Backend API.

3. Install Prerequisites on EC2

SSH into your EC2 instance and run the following commands:

Install Java and Maven

```
sudo apt update
sudo apt install openjdk-17-jdk maven -y
```

Install Docker

```
sudo apt install docker.io -y
sudo usermod -aG docker ubuntu
sudo systemctl enable docker
sudo systemctl start docker
# Note: You may need to log out and log back in for the docker group to take effect.
```

Install & Run SonarQube (via Docker)

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community
```

Access SonarQube at <http://<EC2-IP>:9000>. Default login is *admin / admin*.

Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins -y
sudo usermod -aG docker jenkins # Give Jenkins access to run Docker commands
sudo systemctl restart jenkins
```

Access Jenkins at <http://<EC2-IP>:8080>. Retrieve the initial admin password using `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`.

4. Containerizing the Application

Create a Dockerfile in the root of your backend project directory (backend/bus-tracker-backend/Dockerfile):

```
# Use the official Tomcat image with JDK 17
FROM tomcat:10.1-jdk17

# Remove default Tomcat apps to keep it clean (optional)
RUN rm -rf /usr/local/tomcat/webapps/*

# Copy the generated WAR file into the Tomcat webapps directory
# Tomcat will automatically extract and run the WAR file
COPY target/bus-tracker-backend-1.0.0.war /usr/local/tomcat/webapps/api.war

# Expose the default Tomcat port
EXPOSE 8080

# Start Tomcat
CMD ["catalina.sh", "run"]
```

NOTE: Since we copied the file as `api.war`, your backend API will be accessible at <http://<EC2-IP>:8081/api>. You will need to update the Angular frontend's `environment.prod.ts` to point to this new API URL.

5. Setting up the Jenkins Pipeline

1. In Jenkins, install the following plugins: **Docker Pipeline**, **SonarQube Scanner**.
2. Configure SonarQube in Jenkins:
 - Go to **Manage Jenkins** -> **System** -> Add SonarQube Server details (URL: `http://<EC2-IP>:9000`).
 - Add a SonarQube token in Jenkins Credentials.
3. Create a **New Item** -> **Pipeline** and name it `BusTracker-Deployment`.
4. Use the following `Jenkinsfile` script:

```

pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'bus-tracker-backend:latest'
        SONARQUBE_SERVER = 'SonarQube' // Name configured in Jenkins system
    }

    stages {
        stage('Checkout') {
            steps {
                // Checkout code from your Git repository
                git branch: 'main', url: 'https://github.com/your-repo/bus-tracker.git'
            }
        }

        stage('Build & Unit Test') {
            steps {
                dir('backend/bus-tracker-backend') {
                    sh 'mvn clean package'
                }
            }
        }

        stage('SonarQube Code Analysis') {
            steps {
                dir('backend/bus-tracker-backend') {
                    withSonarQubeEnv(SONARQUBE_SERVER) {
                        sh 'mvn sonar:sonar -Dsonar.projectKey=bus-tracker -Dsonar.projectName="Bus
Tracker"'
                    }
                }
            }
        }

        stage('Quality Gate') {
            steps {
                timeout(time: 1, unit: 'HOURS') {
                    // Wait for SonarQube analysis to complete and pass
                    waitForQualityGate abortPipeline: true
                }
            }
        }

        stage('Docker Build') {
            steps {
                dir('backend/bus-tracker-backend') {
                    sh "docker build -t ${DOCKER_IMAGE} ."
                }
            }
        }

        stage('Deploy to Docker Tomcat') {
            steps {
                // Stop and remove the old container if it exists
                sh "docker stop bus-tracker-api || true"
                sh "docker rm bus-tracker-api || true"

                // Run the new container
                // Mapping host port 8081 to container port 8080 to avoid conflicts with Jenkins
                sh "docker run -d --name bus-tracker-api -p 8081:8080 ${DOCKER_IMAGE}"
            }
        }
    }
}

```

6. How the Pipeline Works

1. **Checkout:** Jenkins pulls the latest code from your Git repository.
2. **Build & Unit Test:** Jenkins runs `mvn clean package`. This compiles your Java code, runs any unit tests, and packages the application into a `.war` file.
3. **SonarQube Analysis:** Maven runs the Sonar scanner, pushing the code quality metrics (bugs, vulnerabilities, code smells) to your SonarQube dashboard.
4. **Quality Gate:** Jenkins waits for SonarQube's assessment. If the code quality fails the defined threshold, the pipeline halts immediately, preventing bad code from being deployed.
5. **Docker Build:** Jenkins uses the `Dockerfile` to create a Docker image that bundles the official Tomcat server with your new `.war` file.
6. **Deploy:** Jenkins spins up the new Docker container on port 8081.

(For the Angular Frontend, you would add similar stages utilizing Node.js, `npm run build`, and deploying the static files via an Nginx Docker container or directly to AWS S3 / CloudFront).